

Arquitecturas Distribuidas en la Industria

1. Descripción del caso

La arquitectura de microservicios es un estilo de diseño donde una aplicación se divide en servicios pequeños e independientes, cada uno con una responsabilidad concreta y que se comunican entre sí mediante mensajes [2]. Este enfoque surgió como respuesta a los problemas de los sistemas tradicionales y fue adoptado en producción por grandes empresas tecnológicas. Uno de los casos más reconocidos es el de **Netflix**: la plataforma de streaming inició en 2009 su migración desde un sistema monolítico, donde todo estaba unido en un solo bloque, hacia esta arquitectura [3].

2. Por qué eligieron este estilo arquitectónico

Los sistemas monolíticos crecen hasta volverse difíciles de manejar: cualquier cambio obliga a reiniciar todo, escalar es costoso y los equipos quedan atados a una sola tecnología [2]. Netflix necesitaba crecer rápido y de forma independiente por partes. Con microservicios, cada módulo funciona de manera autónoma, lo que respondía exactamente a esas necesidades. Netflix ya usaba algo similar bajo el nombre de *fine-grained SOA* antes de que el término microservicios existiera.

3. Beneficios obtenidos

Gracias a esta migración, Netflix logró mayor disponibilidad, tolerancia a fallos y escalabilidad horizontal [3]. Además, este patrón reduce el tiempo de entrega de nuevas funciones y mejora la organización de los equipos. En la práctica, Netflix puede actualizar un servicio sin afectar los demás.

4. Desafíos encontrados

La transición no fue simple. Los principales retos fueron: definir bien los límites de cada servicio, mantener los datos consistentes entre servicios separados, y comprender el sistema como un todo [3]. Además, la arquitectura hereda problemas propios de los sistemas distribuidos, como mayor consumo de recursos.

5. Aplicación al proyecto propio

Nuestro proyecto gestiona solicitudes de soporte técnico de Internet. Funciones como registrar tickets, asignar técnicos y enviar notificaciones podrían separarse en servicios independientes. Si las notificaciones fallan, el registro de tickets seguiría funcionando. En momentos de alta demanda, como una avería masiva, solo se escalaría el módulo necesario. La lección de Netflix aplica directamente: separar responsabilidades produce sistemas más resistentes y fáciles de mantener [3].

Referencias

- [1] A. Balalaie, A. Heydarnoori, y P. Jamshidi, “Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture,” *IEEE Software*, vol. 33, no. 3, pp. 42–52, May./Jun. 2016, doi: 10.1109/MS.2016.64.
- [2] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, y L. Safina, “Microservices: Yesterday, Today, and Tomorrow,” en *Present and Ulterior Software Engineering*, M. Mazzara y B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.
- [3] G. Blinowski, A. Ojdowska, y A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.